

ARCHITETTURE AVANZATE PER I SISTEMI DI ELABORAZIONE E PROGRAMMAZIONE

Costanza Ferrari, Michele De Fusco, Luca Cuconato

Relazione del progetto

Indice

1	Introduzione	2
1.0.1	Come ovviare al problema del costo computazionale . .	2
1.0.2	La distanza approssimata e la quantizzazione binaria .	3
1.0.3	Osservazione sulla natura di $\tilde{d}$	4
2	Ottimizzazioni richieste	5
2.1	Obiettivi del progetto	5
2.2	Varianti architetturali e requisiti di precisione	5
2.2.1	Versione 32-bit SSE (Single Precision)	6
2.2.2	Versione 64-bit AVX (Double Precision)	6
2.3	Ottimizzazioni di basso livello e gestione dati	6
2.3.1	Rappresentazione delle maschere	6
2.3.2	Gestione della dimensionalità e cleanup loop	6
2.3.3	Accessi alla memoria	7
2.3.4	Istruzioni hardware specializzate: PSADBW come po- pulation count vettoriale	7
2.4	Parallelismo di alto livello con OpenMP	7
2.5	Sintesi tecnica delle configurazioni	9
3	Analisi dell'implementazione	10
3.1	Modularità: una logica, quattro motori	10
3.2	Interfacciamento C-Assembly e ABI Windows x64	10
3.3	Il calcolo vettoriale di $\tilde{d}$	12
3.3.1	Loop unrolling e percorso rapido $D = 256$	12
3.4	La distanza euclidea	12
3.5	Implementazione della quantizzazione	13
3.6	Gestione della memoria	13
3.7	Integrazione Python	13

4	Confronto dei tempi rispetto alle ottimizzazioni analizzate	14
4.1	Metodologia	14
4.2	Risultati	15
4.3	Analisi dei risultati	15
4.3.1	Nota metodologica	16
4.4	Conclusioni	16
5	Il pacchetto Python	17
5.1	Interfaccia e moduli	17
5.2	Architettura a tre strati	18
5.3	Gestione della memoria tra i due mondi	19
5.4	Sistema di build	19
5.5	Verifica di correttezza del pacchetto	20

Capitolo 1

Introduzione

La traccia del progetto richiedeva agli studenti, sostanzialmente, di implementare l'algoritmo KNN (K Nearest Neighbors) utilizzato come modello di machine learning e impiegato per la classificazione degli oggetti.

L'obiettivo è quello di trovare, per ogni elemento della collezione Query i k punti più simili presenti nel dataset dei dati.

La ricerca dei k elementi più simili viene fatta sulla base di una misurazione, la distanza euclidea, che va a calcolare la distanza fra vettori prendendo in considerazione ogni componente.

Il problema principale risiede nelle risorse che il calcolo della distanza euclidea, calcolata per ogni vettore presente in Q per tutti gli elementi presenti nel dataset, richiederebbe. Il tempo di esecuzione, sviluppando la distanza euclidea sarebbe $O(n * D * q)$ in cui n sono gli elementi del dataset, q sono gli elementi presenti nella collezione Query e D sono le dimensioni degli elementi nel dataset, le colonne della matrice del dataset.

Valutando che si sta lavorando con collezioni molto estese di dati sarebbe buona norma provare a ridurre per quanto possibile i tempi di esecuzione.

1.0.1 Come ovviare al problema del costo computazionale

Per ridurre i costi la strategia consigliata dalla traccia è quella di fare una sorta di pruning del dataset, quindi di non calcolare la distanza euclidea fra ogni query ed ogni punto del dataset ma di lavorare solo su alcuni di questi punti definiti Pivot. Questa tecnica è utilizzata per ridurre lo spazio di ricerca dei vicini.

I pivot sono un sottoinsieme di punti h selezionati dal dataset che fungono da riferimenti spaziali fissi. L'idea centrale è che, conoscendo la distanza tra un punto del dataset e un pivot, e calcolando la distanza tra la query e lo stesso pivot, è possibile stimare la distanza tra query e punto senza calcolarla effettivamente.

Questa stima si basa sulla disuguaglianza triangolare applicata agli spazi metrici:

$$|d(q, p) - d(v, p)| \leq d(q, v) \quad (1.1)$$

Il termine a sinistra, $|d(q, p) - d(v, p)|$, rappresenta un limite inferiore (lower bound) della distanza reale $d(q, v)$.

Durante la fase di ricerca, se questo limite inferiore risulta superiore alla distanza del k -esimo vicino attualmente più lontano nella lista dei risultati (d_{max}), il punto v può essere scartato immediatamente.

Questa strategia permette di ridurre drasticamente il numero di calcoli esatti in virgola mobile, limitando l'uso delle istruzioni SIMD più onerose (SSE/AVX) solo ai candidati che hanno un'alta probabilità di far parte del set dei vicini più prossimi.

1.0.2 La distanza approssimata e la quantizzazione binaria

Nonostante l'efficacia del pruning tramite pivot, un numero considerevole di punti del dataset potrebbe comunque richiedere una valutazione più approfondita e quindi il calcolo della distanza euclidea.

Per evitare di ricorrere al calcolo della distanza euclidea esatta, la traccia introduce il concetto di distanza approssimata ($\tilde{d}$).

Questa tecnica si basa sulla quantizzazione binaria dei vettori: per ogni vettore $v \in DS$ (e per ogni query q), vengono estratti due vettori binari, v^+ e v^- , di dimensione D . Si individuano le x componenti di v con *valore assoluto* massimo e, per ciascuna di esse:

- se la componente è positiva (o nulla), viene posto a 1 il bit corrispondente di v^+ ;
- se la componente è negativa, viene posto a 1 il bit corrispondente di v^- .

Tutte le altre posizioni restano a 0: dei vettori originali si conservano quindi soltanto la *posizione* e il *segno* delle componenti dominanti.

Grazie a questa rappresentazione, la distanza tra due punti può essere stimata mediante operazioni di prodotto scalare tra vettori binari, estremamente efficienti da calcolare a livello hardware:

$$\tilde{d}(v, w) = (v^+ \cdot w^+) + (v^- \cdot w^-) - (v^+ \cdot w^-) - (v^- \cdot w^+) \quad (1.2)$$

In sintesi, il flusso decisionale per ogni punto v del dataset rispetto a una query q segue una gerarchia di tre livelli:

1. Filtro Pivot: Calcolo del lower bound tramite disuguaglianza triangolare. Se il punto è palesemente lontano, viene scartato.
2. Filtro Approssimato: Se il punto supera il primo filtro, si calcola $\tilde{d}(q, v)$. Se l'approssimazione è peggiore del candidato k -esimo attuale, il punto viene scartato.
3. Verifica Esatta: Solo per i punti superstiti si procede al calcolo della distanza euclidea esatta (Eq. 1) e all'eventuale aggiornamento della lista dei k vicini. Il guadagno in termini di tempi di esecuzione risiede proprio in quest'ultimo passaggio: la formula matematica della distanza euclidea viene effettutata solo su questi ultimi elementi che hanno passato le prime operazioni di filtraggio.

1.0.3 Osservazione sulla natura di $\tilde{d}$

È opportuno segnalare una proprietà non ovvia della misura approssimata. Dalla definizione (Eq. 2) segue che per due vettori identici $\tilde{d}(v, v) = x$ (il valore *massimo*), mentre per vettori con impronte di segno opposte $\tilde{d}$ assume valori negativi: formalmente $\tilde{d}$ si comporta come una *similarità* e non come una *metrica* (non è definita positiva e non soddisfa, in generale, la disuguaglianza triangolare). L'algoritmo previsto dalla traccia la impiega comunque come distanza — selezionando i k punti con $\tilde{d}$ minima e usando $|\tilde{d}(v, p) - \tilde{d}(q, p)|$ come limite inferiore per il pruning — che va quindi inteso come *euristica* conforme alla specifica piuttosto che come filtro matematicamente esatto. I risultati prodotti coincidono al 100% con i file di riferimento ufficiali del progetto, generati con la medesima logica.

Capitolo 2

Ottimizzazioni richieste

2.1 Obiettivi del progetto

L'obiettivo primario dell'attività progettuale risiede nello sviluppo di un'implementazione ad alte prestazioni dell'algoritmo di ricerca dei K vicini (K -Nearest Neighbors). Data la complessità computazionale intrinseca del problema, pari a $O(n \cdot D)$ per query, il progetto si focalizza sul superamento dei colli di bottiglia del codice seriale attraverso l'uso di tecniche di ottimizzazione hardware-aware, sfruttando il calcolo vettoriale (SIMD) e il parallelismo multi-thread.

Un principio guida ha orientato tutte le scelte: **ottimizzare dove il tempo si consuma davvero**. Grazie ai filtri descritti nel Capitolo 1, l'operazione dominante del programma non è la distanza euclidea (calcolata solo per i k finalisti di ogni query, circa 16.000 volte in tutto), bensì la distanza approssimata $\tilde{d}$ sulle maschere binarie, invocata milioni di volte durante la scansione del dataset. È dunque su $\tilde{d}$ che si concentrano la vettorizzazione e la scrittura in Assembly.

2.2 Varianti architetturali e requisiti di precisione

La traccia impone la realizzazione di due soluzioni distinte, differenziate per architettura di riferimento e precisione numerica, al fine di confrontare l'impatto delle diverse estensioni del set di istruzioni x86.

2.2.1 Versione 32-bit SSE (Single Precision)

- **Precisione:** Utilizzo del tipo di dato `float` (32 bit) per dataset, query e distanze euclidee.
- **Set di istruzioni:** Repertorio **SSE2** (*Streaming SIMD Extensions*).
- **Parallelismo hardware:** Registri **XMM** da 128 bit. Poiché le maschere di quantizzazione sono vettori di byte (un byte per dimensione, valori 0/1), ogni istruzione vettoriale elabora **16 elementi della maschera per ciclo**.

2.2.2 Versione 64-bit AVX (Double Precision)

- **Precisione:** Utilizzo del tipo di dato `double` (64 bit).
- **Set di istruzioni:** Repertorio **AVX2** (*Advanced Vector Extensions*).
- **Parallelismo hardware:** Registri **YMM** da 256 bit: **32 byte di maschera per istruzione** nel calcolo di $\tilde{d}$; nella versione a intrinseci, la distanza euclidea su `double` viene vettorizzata elaborando 4 elementi per ciclo.

2.3 Ottimizzazioni di basso livello e gestione dati

2.3.1 Rappresentazione delle maschere

Le quantizzazioni v^+ e v^- sono memorizzate come array di `uint8_t` con valori 0/1: un byte per dimensione. La scelta è deliberata: sui byte l'hardware SIMD dispone di operazioni logiche e di somma orizzontale estremamente efficienti, e l'intero corredo di maschere del dataset ($2 \times 2000 \times 256$ byte $\approx$ 1 MB) più la matrice dell'indice (128 KB) risiede stabilmente nelle cache di livello 2/3.

2.3.2 Gestione della dimensionalità e cleanup loop

Il loop principale vettorizzato elabora blocchi interi di registro (16 byte in SSE2, 32 in AVX2). Poiché non è garantito che D sia un multiplo esatto

della dimensione di blocco, al termine del loop una sezione di *cleanup* scalare processa le componenti residue, garantendo il risultato corretto per qualunque D ed evitando accessi a memoria non allocata. Per il caso di riferimento $D = 256$, inoltre, le routine Assembly dispongono di un percorso dedicato completamente srotolato (si veda il Capitolo 3).

2.3.3 Accessi alla memoria

Le routine vettoriali impiegano **load non allineate** (MOVDQU in SSE2, VMOVDQU in AVX2). La scelta elimina qualunque vincolo di allineamento sui buffer delle maschere, semplificando l'interfacciamento con il resto del programma (e con NumPy, nel pacchetto Python): sulle microarchitetture correnti la penalità delle load non allineate su dati residenti in cache è trascurabile, mentre un vincolo di allineamento avrebbe imposto allocatori dedicati su tutta la filiera.

2.3.4 Istruzioni hardware specializzate: PSADBW come population count vettoriale

Il cuore delle routine ottimizzate è l'istruzione PSADBW (*Packed Sum of Absolute Differences of Bytes*), nata per la stima di moto nei codec video: calcola la somma delle differenze assolute tra i byte di due registri. Usata contro il registro *zero*, la “differenza assoluta da zero” di un byte è il byte stesso: PSADBW diventa così una **somma orizzontale di 16 (o 32) byte in una singola istruzione**. Su maschere di valori 0/1, sommare i byte equivale a *contare gli 1*: ogni prodotto scalare della definizione di $\tilde{d}$ si riduce a una PAND (intersezione dei bit) seguita da una PSADBW (conteggio).

Si è scelto consapevolmente di **non** impiegare l'istruzione scalare POPCNT: avrebbe richiesto di trasferire i dati dai registri vettoriali a quelli general-purpose a ogni blocco, mentre PSADBW mantiene l'intero conteggio nel dominio SIMD, accumulando con PADDQ direttamente nei registri XMM/YMM.

2.4 Parallelismo di alto livello con OpenMP

Per entrambe le soluzioni è stata realizzata una variante **OpenMP**. Mentre la vettorizzazione accelera il calcolo della singola distanza (parallelismo a livello di dati), OpenMP introduce il parallelismo a livello di thread distribuendo il

loop delle query tra i core della CPU. Il problema è ideale per questa strategia: ogni query legge strutture in sola lettura (dataset, indice, maschere) e scrive esclusivamente nel proprio segmento dell'array dei risultati — nessuna sincronizzazione è necessaria. Si è adottato `schedule(dynamic)` perché il pruning rende il costo delle query disomogeneo: l'assegnazione dinamica evita che i thread restino inattivi in attesa dei blocchi più lenti.

2.5 Sintesi tecnica delle configurazioni

La tabella seguente riassume le specifiche tecniche delle implementazioni ottimizzate, quali risultano dal codice consegnato.

Caratteristica	Versione SSE2	Versione AVX2
Precisione	Single Precision (32-bit)	Double Precision (64-bit)
Tipo di dato C	float	double
Registro SIMD	XMM (128 bit)	YMM (256 bit)
Byte di maschera per istruzione	16	32
Istruzioni chiave ($\tilde{d}$)	MOVDQU, PAND, PSADBW, PADDQ	VMOVDQU, VPAND, VPSADBW, VPADDQ
Accessi a memoria	load non allineate	load non allineate
Parallelismo	Data-level (SI- MD) + Thread- level (variante OpenMP)	Data-level (SI- MD) + Thread- level (variante OpenMP)

Tabella 2.1: Confronto tecnico tra le architetture target del progetto.

Capitolo 3

Analisi dell'implementazione

In questo capitolo viene analizzato il codice sorgente del progetto, evidenziando le scelte tecniche adottate per garantire l'efficienza computazionale e il corretto interfacciamento tra i moduli in C e le routine in Assembly.

3.1 Modularità: una logica, quattro motori

La scelta architetturale più importante del progetto è la separazione tra la *logica* dell'algoritmo e il *motore* di calcolo della distanza approssimata. I moduli di alto livello — `index.c` (costruzione dell'indice), `query.c` e `query64.c` (ricerca dei vicini) — esistono in un'unica versione e non contengono alcun codice dipendente dall'architettura; tutte le varianti (scalare, intrinseci, Assembly) si ottengono ricompilando gli stessi sorgenti con macro di preprocessore diverse:

Questo approccio garantisce che il confronto prestazionale del Capitolo 4 misuri esclusivamente l'effetto dell'ottimizzazione della funzione critica: tutto il resto del programma è, letteralmente, lo stesso codice. La versione scalare funge inoltre da riferimento (*baseline*) per la validazione dei risultati.

3.2 Interfacciamento C–Assembly e ABI Windows x64

Le routine Assembly (`distance_sse2.S`, `distance_avx2.S`, sintassi Intel assemble da GCC) espongono una firma C convenzionale:

File	Macro	Implementazione di <code>approximate_distance()</code>
<code>distance.c</code>	—	ciclo scalare C sui byte delle maschere
<code>distance.c</code>	<code>USE_SSE2</code>	intrinseci SSE2 (registri XMM)
<code>distance.c</code>	<code>USE_AVX</code>	intrinseci AVX2 (registri YMM)
<code>distance32ASSEMBLY.c</code>	<code>USE_SSE2_ASM</code>	chiamata alla routine esterna <code>approximate_distance_sse2_asm</code>
<code>distance64ASSEMBLY.c</code>	<code>USE_AVX_ASM</code>	chiamata alla routine esterna <code>approximate_distance_avx2_asm</code>

Tabella 3.1: Commutazione del back-end tramite macro.

```
int approximate_distance_xxx_asm(const uint8_t *vp, const uint8_t *vn,
                                const uint8_t *wp, const uint8_t *wn,
                                size_t D);
```

Secondo la convenzione di chiamata **Windows x64**, i quattro puntatori alle maschere arrivano nei registri RCX, RDX, R8 e R9, mentre il quinto argomento D viene letto dallo stack (all'ingresso si trova a `[RSP+40]`, oltre l'indirizzo di ritorno e i 32 byte di *shadow space*; l'offset effettivo usato nel codice tiene conto dello spazio allocato dal prologo).

Il rispetto dell'ABI è curato con attenzione, requisito indispensabile per una funzione invocata milioni di volte anche da thread OpenMP concorrenti:

- nel **prologo** vengono salvati sullo stack i registri non volatili utilizzati (nella versione SSE2: XMM6--XMM11 e, tramite `push, r12--r15`); nell'**epilogo** vengono ripristinati nell'ordine inverso;
- la routine AVX2 termina con `VZEROUPPER`, che azzerla la parte alta dei registri YMM evitando le penalità di transizione tra codice AVX e codice SSE legacy;
- il valore di ritorno (un intero, la distanza approssimata) viene restituito in RAX.

3.3 Il calcolo vettoriale di $\tilde{d}$

Per ogni blocco di maschera (16 byte in SSE2, 32 in AVX2) ciascuno dei quattro termini di $\tilde{d}$ viene calcolato con lo schema descritto nel Capitolo 2. Nella versione AVX2 bastano tre istruzioni per termine:

```
vpand    ymm4, ymm0, ymm2    ; v+ AND w+ : byte a 1 dove entrambi hanno 1
vpsadbw  ymm4, ymm4, ymm15    ; somma orizzontale contro zero = conteggio
vpaddq   ymm8, ymm8, ymm4     ; accumulo del termine pp
```

La versione SSE2 aggiunge, prima del conteggio, una normalizzazione difensiva (PCMPEQB/PXOR/PAND con una costante di 1) che rende ogni byte non nullo esattamente pari a 1: il codice risulterebbe corretto anche con maschere non strettamente binarie. La versione AVX2 omette questo passaggio, affidandosi al contratto — garantito dalla quantizzazione — che i byte valgano solo 0 o 1: due filosofie entrambe legittime, la seconda più rapida di tre istruzioni per termine.

Al termine del loop i quattro accumulatori vengono ridotti a scalari e combinati come $(pp + nn) - (pn + np)$, con il risultato depositato in **RAX**.

3.3.1 Loop unrolling e percorso rapido $D = 256$

Poiché il carico di lavoro di riferimento ha $D = 256$, entrambe le routine controllano questo caso all'ingresso (`cmp r10, 256`) e deviano su un percorso **completamente srotolato**: 16 blocchi da 16 byte in SSE2, 8 blocchi da 32 byte in AVX2, scritti in sequenza senza istruzioni di controllo del ciclo. L'eliminazione di incrementi, confronti e salti condizionati mantiene la pipeline costantemente alimentata; è questo il percorso eseguito nella totalità delle chiamate del benchmark. Per ogni altro valore di D resta disponibile il loop generico con cleanup scalare dei resti.

3.4 La distanza euclidea

Grazie all'architettura a filtri, la distanza euclidea esatta viene calcolata soltanto per i k finalisti di ogni query: con i parametri di riferimento sono $8 \times 2000 = 16.000$ chiamate, contro i circa 2,4 milioni di valutazioni di $\tilde{d}$. Per questa ragione non è stata realizzata una versione Assembly dell'euclidea:

nelle build Assembly essa è un semplice ciclo scalare C, mentre nella build a intrinseci AVX a 64 bit viene vettorizzata con `_mm256_sub_pd/_mm256_mul_pd` (4 `double` per iterazione). L'effetto sul tempo totale è in ogni caso marginale: l'ottimizzazione è stata concentrata dove risiede il collo di bottiglia.

3.5 Implementazione della quantizzazione

La quantizzazione è implementata interamente in C (`quantization.c`), in due versioni gemelle per `float` e `double`. Per ogni vettore viene costruito un array ausiliario di coppie $(|v_i|, i)$, ordinato in senso decrescente con `qsort`; i bit di v^+ o v^- corrispondenti alle prime x posizioni vengono quindi accesi in base al segno della componente originale. Il costo è $O(D \log D)$ per vettore.

La scelta di non vettorizzare questa fase è motivata dai numeri: la quantizzazione dell'intero dataset avviene una sola volta nella fase di build (2000 vettori, decine di millisecondi), e a query time se ne esegue una sola per query. Sui dataset del progetto si è inoltre verificata l'assenza di pareggi di valore assoluto al confine di selezione: l'esito della quantizzazione è pertanto deterministico e indipendente dai dettagli dell'ordinamento.

3.6 Gestione della memoria

Il modulo `matrix.c` carica i file `.ds2` (header di due `uint32` con n e D , seguito dai dati row-major) in buffer allocati con `malloc` standard. Non è imposto alcun vincolo di allineamento: come discusso nel Capitolo 2, le routine vettoriali usano load non allineate, quindi nessun allocatore speciale è necessario nella filiera C. L'indice (`Index`) possiede tutte le proprie strutture — maschere del dataset, maschere dei pivot, matrice delle distanze — e viene rilasciato con un'unica `free_index()`; ogni percorso di errore nella costruzione rilascia le allocazioni parziali prima di ritornare `NULL`.

3.7 Integrazione Python

L'intero sistema è inoltre fruibile come pacchetto Python nativo, con la stessa base di codice C e Assembly qui descritta. All'argomento, per la sua estensione, è dedicato il Capitolo 5.

Capitolo 4

Confronto dei tempi rispetto alle ottimizzazioni analizzate

In questo capitolo vengono analizzati i risultati dei test prestazionali, confrontando le otto configurazioni dell'algoritmo K-NN.

4.1 Metodologia

Tutte le misure si riferiscono al carico di lavoro di riferimento del progetto: dataset di $n = 2000$ punti a $D = 256$ dimensioni, 2000 query, $h = 16$ pivot, $k = 8$ vicini, quantizzazione $x = 64$. I tempi sono suddivisi in **Build** (quantizzazione del dataset e costruzione dell'indice) e **Query** (ricerca dei k vicini per tutte le query, ossia l'algoritmo oggetto dell'ottimizzazione).

Le esecuzioni sono automatizzate dal launcher `mainReport.c` (target `Benchmark_Report`), che lancia in sequenza gli otto eseguibili, ne analizza l'output e genera la tabella riassuntiva (`report_completo.txt`) e un report HTML con l'evidenziazione dei tempi migliori. Nelle configurazioni OpenMP il tempo è misurato con `omp_get_wtime()` (tempo di parete) anziché con `clock()`: in un programma multi-thread quest'ultima può sommare il tempo CPU di tutti i thread, falsando il confronto.

4.2 Risultati

Configurazione	Build (ms)	Query (ms)	Totale (ms)	Speedup query
32-bit Scalar (C)	84	4352	4436	1×
32-bit SSE2 (intrinseci)	82	2357	2439	1,8×
32-bit SSE2 (Assembly)	68	334	402	13,0×
32-bit SSE2 + OpenMP	88	88	176	49,5×
64-bit Scalar (C)	193	4840	5033	1×
64-bit AVX2 (intrinseci)	85	2542	2627	1,9×
64-bit AVX2 (Assembly)	67	706	773	6,9×
64-bit AVX2 + OpenMP	67	109	176	44,4×

Tabella 4.1: Tempi misurati sulla macchina di sviluppo (valori di `report_completo.txt`). Lo speedup è calcolato sulla fase di query rispetto alla versione scalare della stessa precisione.

4.3 Analisi dei risultati

1. **Il salto prestazionale maggiore è dovuto all'Assembly.** Nella versione a 32 bit la query passa da 4352 ms a 334 ms (13×), a 64 bit da 4840 ms a 706 ms (6,9×). Il merito principale è dello schema PAND+PSADBW (Capitolo 3), che riduce ogni prodotto scalare tra maschere a poche istruzioni vettoriali senza mai uscire dal dominio SIMD, unito al percorso srotolato per $D = 256$.
2. **OpenMP moltiplica il guadagno quasi linearmente.** La distribuzione delle query sui core porta la fase di query a 88 ms (32 bit) e 109 ms (64 bit): rispetto alle versioni scalari lo speedup complessivo è di circa 49× e 44×. L'assenza di sincronizzazioni (query indipendenti, strutture in sola lettura) rende il problema ideale per il parallelismo a thread; `schedule(dynamic)` compensa il costo disomogeneo delle query indotto dal pruning.
3. **Dopo l'ottimizzazione, il collo di bottiglia si sposta.** Nelle configurazioni OpenMP la fase di query costa quanto (o meno) della fase

di build (88 vs 88 ms; 109 vs 67 ms): abbattuto il termine dominante, emergono i costi prima trascurabili — i 2000 ordinamenti della quantizzazione e il caricamento dei dati. È il comportamento previsto dalla legge di Amdahl, e indica che ulteriori ottimizzazioni dovrebbero rivolgersi alla fase di build.

4. **Il divario tra 32 e 64 bit nella versione Assembly** (334 vs 706 ms) non dipende dal calcolo di $\tilde{d}$, identico nei due casi (le maschere sono byte in entrambi): vi concorrono la scansione di dati a doppia ampiezza (banda di memoria), il ricalcolo euclideo su `double` e i costi di gestione dei registri YMM. Con OpenMP il divario quasi si annulla (109 vs 88 ms).

4.3.1 Nota metodologica

Per correttezza va segnalato che i target Scalar e a intrinseci sono compilati senza flag di ottimizzazione, mentre i target Assembly e OpenMP usano `-O3`: il confronto tra intrinseci e Assembly sovrastima quindi in parte il vantaggio di quest'ultimo (a `-O0` gli intrinseci generano codice con traffico di stack a ogni operazione). Il dato non inficia la conclusione qualitativa — lo schema PSADBW resta superiore alla catena di unpack e somme a 16 bit degli intrinseci — ma i rapporti tra le colonne centrali della tabella vanno letti con questa avvertenza.

4.4 Conclusioni

I test confermano l'efficacia dell'architettura a tre livelli (filtro pivot, filtro approssimato, verifica esatta) descritta nel Capitolo 1: sui dati di riferimento il pruning elimina il 39% delle valutazioni complete e riduce le distanze euclidee esatte da 4 milioni a 16.000 (250 volte di meno). Sul piano architetturale, lo stesso identico algoritmo passa da 4,4 a 0,18 secondi complessivi — quasi 50× sulla fase di query — senza modificare una riga della logica: il guadagno proviene interamente dalla discesa di livello dove il tempo si consuma (la distanza approssimata, ridotta a PAND+PSADBW) e dalla salita di livello dove il problema lo consente (un `pragma` OpenMP sul loop delle query).

Capitolo 5

Il pacchetto Python

Come richiesto dalla traccia, l'intero sistema è reso disponibile come pacchetto Python nativo, denominato `Gruppo_Ferrari_DeFusco_Cuconato`. Il requisito fondante della progettazione è stato **non duplicare l'algoritmo**: il pacchetto non reimplementa nulla, ma espone attraverso le Python C-API lo stesso identico codice C e Assembly validato nei capitoli precedenti. Ne consegue che ogni proprietà dimostrata per gli eseguibili — correttezza rispetto ai riferimenti ufficiali inclusa — vale automaticamente anche per la libreria.

5.1 Interfaccia e moduli

Il pacchetto espone tre sotto-moduli, uno per configurazione hardware, ciascuno con la medesima classe `QuantPivot` dotata di un'interfaccia in stile *scikit-learn*:

Modulo	Precisione	Back-end
<code>quantpivot32</code>	<code>float32</code>	Assembly SSE2
<code>quantpivot64</code>	<code>float64</code>	Assembly AVX2
<code>quantpivot64omp</code>	<code>float64</code>	Assembly AVX2 + OpenMP

Tabella 5.1: I tre moduli del pacchetto.

Il metodo `fit(dataset, n_pivots, quant_level)` costruisce l'indice a pivot (quantizzazione del dataset e matrice $\tilde{d}(v, p)$) e restituisce l'oggetto

stesso, consentendo il concatenamento; `predict(query, k)` esegue la ricerca dei k vicini per tutte le query e restituisce la tupla (`ids`, `dists`): due array NumPy di forma (n_q, k) contenenti gli identificativi dei vicini e le rispettive distanze euclidee reali.

```
import numpy as np
from Gruppo_Ferrari_DeFusco_Cuconato.quantpivot64omp import QuantPivot

DS = np.ascontiguousarray(dataset, dtype=np.float64)    # (N, D)
Q  = np.ascontiguousarray(query, dtype=np.float64)      # (nq, D)

model      = QuantPivot().fit(DS, n_pivots=16, quant_level=64)
ids, dists = model.predict(Q, k=8)
```

5.2 Architettura a tre strati

L'integrazione è organizzata in tre strati, dal più esterno al più interno:

1. **Wrapper C-API** (`quantpivot*_py.c`): definisce il tipo Python `QuantPivot` e i suoi metodi. Riceve gli array NumPy, ne verifica forma (bidimensionale), tipo (`float32/float64` a seconda del modulo) e allineamento, quindi ne passa al livello inferiore i puntatori ai dati grezzi: **nessuna copia** dei dati viene mai effettuata.
2. **Adattatore** (`quantpivot32.c`, `quantpivot64.c`, `quantpivot64omp.c`): traduce la struttura ponte `params` nelle strutture native del nucleo (`MatrixF32/MatrixF64`, `Index`, `Neighbor`) e invoca `build_index()` in `fit` e `knn_query_all()` in `predict`.
3. **Nucleo condiviso**: gli stessi `index.c`, `query.c/query64.c`, `quantization.c` e le routine Assembly degli eseguibili, compilati con le medesime macro (`USE_SSE2_ASM` o `USE_AVX_ASM`, più `-fopenmp` per la variante parallela).

La struttura `params`, definita in `include/common.h`, è l'unico oggetto che attraversa il confine tra gli strati: raccoglie i puntatori a dataset e query, i parametri del problema (h , k , x , dimensioni), il puntatore all'indice costruito e i buffer dei risultati. La macro `QP_DOUBLE` commuta il tipo scalare `type` (`float/double`) e la costante di allineamento `align` (16/32 byte), permettendo di condividere lo stesso wrapper tra le versioni a diversa precisione.

5.3 Gestione della memoria tra i due mondi

La coesistenza del garbage collector di Python con le allocazioni manuali del C richiede alcune cautele, tutte gestite nel wrapper:

- **Riferimenti agli input:** `fit` e `predict` incrementano il contatore di riferimenti (`Py_INCREF`) degli array NumPy ricevuti e lo rilasciano solo alla distruzione dell'oggetto: gli array non possono essere deallocati da Python mentre il C ne sta usando i puntatori.
- **Proprietà dei risultati:** i buffer di `ids` e `dists` sono allocati in C (`_mm_malloc`) e “adottati” dagli array NumPy restituiti tramite `PyCapsule` con distruttore: quando l'ultimo riferimento Python all'array cade, il distruttore libera la memoria C. Non esistono quindi né copie né perdite di memoria.
- **Distruzione del modello:** alla deallocazione dell'oggetto `QuantPivot`, l'indice viene rilasciato con la stessa `free_index()` usata dagli eseguibili.

5.4 Sistema di build

Il pacchetto si compila con `setuptools` (`python/setup.py + pyproject.toml`), che dichiara tre estensioni native — una per modulo — ciascuna composta dai propri sorgenti C e dal file *Assembly .S* corrispondente, con le macro e i flag appropriati (`-O3 -msse2 / -O3 -mavx2`, più `-fopenmp` per la variante parallela).

Due aspetti hanno richiesto una classe `build_ext` personalizzata:

- `distutils` non riconosce i sorgenti *.S*: l'estensione va registrata esplicitamente tra i tipi compilabili;
- su Windows il compilatore predefinito sarebbe MSVC, che non è in grado di assemblare i file in sintassi GAS né accetta i flag GCC: la classe forza l'uso di **MinGW-w64** (GCC).

Il wheel prodotto viene infine reso *autosufficiente* con lo strumento `delvewheel`, che vi incorpora le DLL runtime di GCC (`libgcc`, `libgomp`, `libwinpthread`): l'utente finale installa il pacchetto con un semplice `pip install` senza necessità di avere MinGW sul proprio sistema.

5.5 Verifica di correttezza del pacchetto

La libreria è stata sottoposta alla stessa verifica degli eseguibili: per ciascuno dei tre moduli, `fit + predict` sul carico di lavoro di riferimento ($n = 2000$, $D = 256$, $h = 16$, $k = 8$, $x = 64$) riproducono **2000 query su 2000** dei file di riferimento ufficiali — stessi identificativi nello stesso ordine e stesse distanze (tolleranza 10^{-3} per `float32`, 10^{-9} per `float64`). Il percorso verificato attraversa l'intera catena: validazione NumPy, wrapper C-API, adattatore, nucleo C e routine Assembly (con OpenMP attivo nella terza variante).

Lo script dimostrativo `examples/esempio_knn.py` ripete questa verifica a ogni esecuzione, riportando anche i tempi: sul carico di riferimento la variante `quantpivot64omp` completa la fase di `predict` in circa 60–70 ms, coerentemente con i risultati del Capitolo 4. Il beneficio del multi-threading resta dunque pienamente fruibile anche dall'interno dell'interprete Python: durante l'esecuzione delle routine native il GIL non costituisce un ostacolo, poiché il parallelismo è gestito interamente da OpenMP nello strato C.